



UNIVERSITÀ
DEGLI STUDI
FIRENZE

Ingegneria del Software

SunSpot: Gestionale per la prenotazione digitale di stabilimenti
balneari

Soldani Leonardo - 7139732
Neamtu Narcis Daniel - 7133414

Indice

1	Introduzione	1
2	Ambiente e tecnologie utilizzate	1
2.1	Linguaggio e Tool di Build	1
2.2	Database e Persistenza	1
2.3	Librerie e Containerizzazione	1
2.4	Testing e Analisi	1
2.5	Intelligenza artificiale	1
3	Use Cases	2
3.1	Utenti del sistema	2
3.2	Descrizione Use Cases	2
3.2.1	Manage User Account	4
3.2.2	Create Booking	5
3.2.3	Manage Reviews	6
3.2.4	Create Beach	7
3.2.5	Manage Beach	8
4	Progettazione	8
4.1	Pattern Architeturali e Progettuali	8
4.1.1	Architettura Esagonale (Ports & Adapters)	9
4.1.2	DDD Light (Domain-Driven Design)	9
4.1.3	Builder Pattern	10
4.2	Struttura del codice	10
4.3	Domain	11
4.4	Application	11
4.5	Infrastructure	11
5	Database	12
5.1	Modello ER	12
5.2	Dizionario dei dati	12
5.3	Tabelle delle Regole	14
6	Mockup	16
7	Testing	16

1 Introduzione

Il progetto SunSpot nasce dall'esigenza di digitalizzare e modernizzare il settore degli stabilimenti balneari, un ambito che ancora oggi mostra una certa resistenza all'adozione di piattaforme web integrate. L'obiettivo principale è fornire un'infrastruttura capace di permettere all'utente di prenotare in modo rapido e intuitivo un ombrellone presso uno dei lidi aderenti al servizio, trasformando la pianificazione di una giornata al mare in un'operazione a portata di click, offrendo al contempo ai gestori uno strumento completo per il controllo della propria attività.

Per garantire un ecosistema sicuro e affidabile, SunSpot integra un sistema di regolazione basato su recensioni verificate e un meccanismo di segnalazione (report) tra utenti e gestori, volto a preservare l'integrità del servizio e a migliorare costantemente l'esperienza d'uso per tutti gli attori coinvolti.

2 Ambiente e tecnologie utilizzate

L'applicazione è stata sviluppata utilizzando le seguenti tecnologie:

2.1 Linguaggio e Tool di Build

- **Java 25** - Scelto per l'utilizzo delle funzionalità più recenti della JDK.
- **Maven 3.8.5** - Strumento di build e gestione delle dipendenze.

2.2 Database e Persistenza

- **MySQL 8.3.0** - Database relazionale utilizzato per la persistenza dei dati.
- **MySQL Connector/J** - Driver ufficiale per la gestione della connessione al database.

2.3 Librerie e Containerizzazione

- **BCrypt 0.10.2** - Utilizzata per l'hashing e la gestione sicura delle password.
- **Docker & Docker Compose 3.8** - Utilizzati per containerizzare il database MySQL, garantendo un ambiente di esecuzione isolato, persistente e facilmente riproducibile.

2.4 Testing e Analisi

- **JUnit 5.12.1** - Framework per l'esecuzione dei test unitari.
- **Mockito 5.23.0** - Utilizzato per la simulazione di dipendenze durante i test (mocking).
- **JUnit Platform Suite** - Strumento per l'organizzazione e l'esecuzione dei test.

2.5 Intelligenza artificiale

A supporto dello sviluppo del progetto sono stati utilizzati i modelli linguistici *Gemini 3* e *Gemini 3.1*. L'impiego di tali strumenti si è concentrato sui seguenti ambiti:

- **Architettura progetto:** una volta definiti gli UML alla base del nostro progetto abbiamo consultato i modelli per un confronto critico sui pattern architetturali (sezione 4) più idonei al nostro contesto applicativo.
- **Generazione codice ripetitivo:** al fine di ottimizzare i tempi di sviluppo, l'intelligenza artificiale è stata utilizzata per la generazione di codice strutturale e ripetitivo (es. pattern *Builder*). Ha inoltre fornito spunti per soluzioni alternative in componenti critici, come nel

caso del *TransactionManager*. L'utilizzo è sempre stato posto sotto controllo umano al fine di garantire l'integrità del codice e la corretta organizzazione di quest'ultimo.

- **Testing:** i modelli sono stati impiegati come supporto per la stesura dei casi di test e per una verifica finale sulla copertura e completezza della suite di testing, assicurando una maggiore robustezza al software prodotto.

3 Use Cases

3.1 Utenti del sistema

Il programma coinvolge tre attori principali: Customer, Owner e Admin. Ciascuno di essi ha la possibilità di effettuare azioni specifiche per il proprio ruolo:

- **Customer:** utente finale dell'applicazione, ha la possibilità di effettuare prenotazioni scegliendo l'ombrellone desiderato; può richiedere inoltre servizi extra come lettini, sdraio e sedie, o la prenotazione di un camerino. Questa tipologia di utente può rilasciare un'unica recensione per ogni spiaggia presso cui ha prenotato e può inviare un report (segnalazione) agli admin qualora riscontri violazioni delle linee guida da parte di uno stabilimento.
- **Owner:** gestore dello stabilimento balneare, ha il compito di configurare il proprio lido partendo dalla segnalazione dei servizi comuni, passando per l'organizzazione geografica della spiaggia in zone con diverse fasce di prezzo, fino alla possibilità di definire stagionalità con diverse tariffe. Può inoltre effettuare un report nei confronti dei customer che hanno tenuto comportamenti scorretti, come ad esempio il mancato rispetto di una prenotazione.
- **Admin:** è colui che svolge il ruolo di gestore dell'ambiente commerciale offerto dall'applicazione, accogliendo i report e valutandoli tramite l'accesso alla cronologia dei report inviati e ricevuti dagli utenti, con la possibilità di accettarli o rifiutarli. Al fine di gestire l'ambiente, l'admin utilizza lo strumento del ban, che può colpire sia i customer che gli owner/spiagge. Il ban può essere di due tipi: da una singola spiaggia (solo per i customer) oppure dall'intera applicazione (per entrambi).

3.2 Descrizione Use Cases

Il programma si pone l'obiettivo di implementare un'applicazione a scopi commerciali, di conseguenza l'azione di eliminare prenotazioni, spiagge, utenti ed altre informazioni di natura fiscale risulterebbe un comportamento superficiale da parte dei programmatori. Per questo motivo, il sistema non prevede l'eliminazione fisica di record sensibili, ma utilizza un meccanismo di disattivazione logica.

Di seguito (fig. 1) è riportato il diagramma degli Use Cases del progetto, nelle sezioni successive verranno analizzati nel dettaglio i flussi più rilevanti.

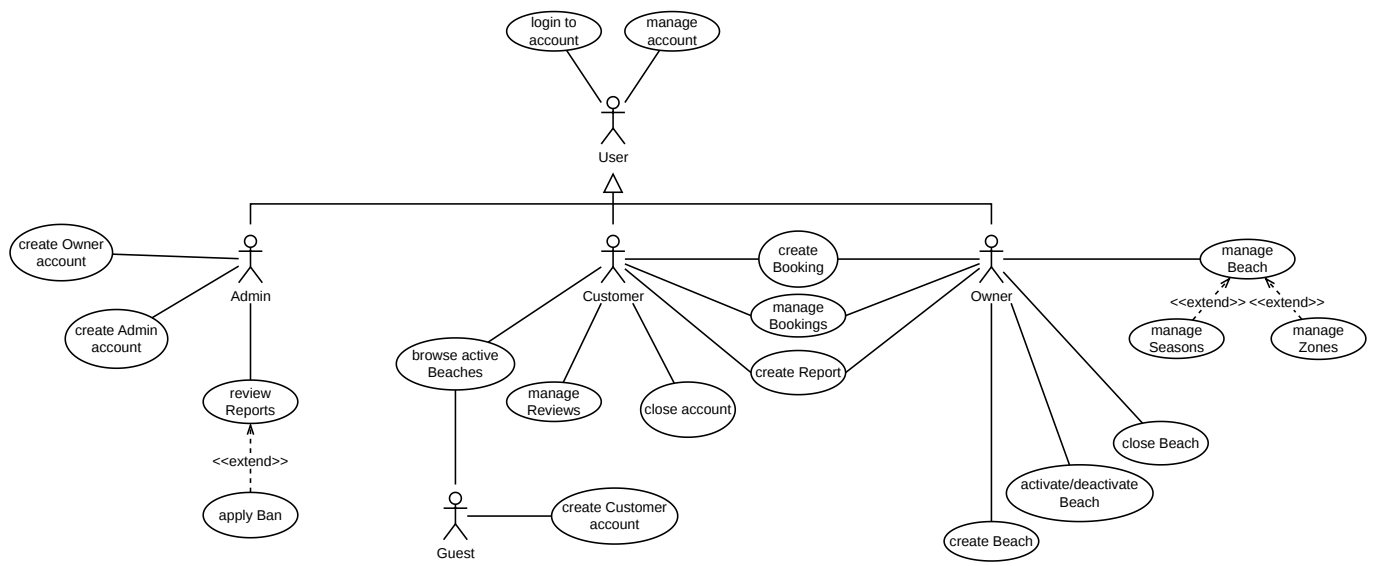


Figura 1: Diagramma degli Use Cases

3.2.1 Manage User Account

ID Use Case	UC-02
Nome Use Case	Manage Account
Attori	User (Admin, Customer, Owner)
Descrizione	Permette all'utente di modificare i suoi dati legati all'account (username, email, numero di telefono, indirizzo ecc.).
Precondizioni	L'utente in questione deve aver fatto login inizialmente all'account.
Postcondizioni	I dati modificati vengono aggiornati nel database e viene mostrato all'utente un messaggio di conferma dei cambiamenti.
Flusso Principale	1. L'utente naviga dentro la pagina "Impostazioni Account/Modifica informazioni"; 2. Il sistema recupera i dati e mostra i dati attuali dell'account (nome, cognome); 3. L'utente modifica i dati che preferisce; 4. L'utente clicca su "Salva modifiche"; 5. Il sistema convalida e salva i nuovi dati; 6. Viene mosrato all'utente un messaggio di conferma.
Flussi alternativi	2a. L'utente è un Customer: Il sistema mostra anche l'indirizzo; 3a. L'utente è un Customer L'utente può modificare anche l'indirizzo; 1a. Cambio email (da passo 1): i. L'utente naviga su "Cambia Email"; ii. Il sistema mostra una pagina dedicata per il cambio email; iii. L'utente inserisce la nuova email e la password attuale; iv. Il sistema verifica l'unicità della email nel database e aggiorna la nuova email nel database. 1b-e. Email già usata per un altro account/non valida: Il sistema segnala l'errore all'utente e invita a ricontrollare. 1b-e. Password non valida: Il sistema segnala l'errore all'utente e invita a reinserire la password. 1c. Cambio numero di telefono (L'utente è un Customer, da passo 1): i. L'utente naviga su "Cambia numero di telefono"; ii. Il sistema mostra una pagina dedicata per il cambio numero di telefono; iii. L'utente inserisce il nuovo numero di telefono e la password attuale; iv. Il sistema verifica l'unicità del numero nel database e aggiorna il nuovo numero nel database. 1c-e. Numero di telefono già usato per un altro account/non valido: Il sistema segnala l'errore all'utente e invita a ricontrollare. 1c-e. Password non valida: Il sistema segnala l'errore all'utente e invita a reinserire la password. 1d. Cambio password (da passo 1): i. L'utente naviga su "Cambia password"; ii. Il sistema mostra una pagina dedicata per il cambio password; iii. L'utente inserisce la nuova e la vecchia password; iv. Il sistema verifica la password vecchia; v. Il sistema aggiorna la password nel database e mostra un messaggio di conferma all'utente. 1d-e. Password vecchia non valida: Il sistema segnala l'errore all'utente e invita a reinserire la vecchia password.
Business Rules	• Ogni utente deve inserire dati validi. I dati non possono essere nulli. • Solo Customer possiede indirizzo e numero di telefono; • Username, nome, cognome (per i Customers: indirizzo) possono essere cambiati allo stesso momento. Email, Password (per i Customers: numero di telefono) invece devono essere cambiati singolarmente attraversando dei check di sicurezza.

Figura 2: UC: Manage Account

3.2.2 Create Booking

ID Use Case	UC-05
Nome Use Case	Create Booking
Attori	Customer, Owner
Descrizione	Permette ad un Customer di prenotare posti, servizi extra e parcheggi presso una spiaggia in una certa data. Permette inoltre ad un Owner di registrare manualmente una prenotazione per un cliente telefonico.
Precondizioni	L'utente deve aver effettuato il login. Se l'utente è un Customer, non deve essere bannato dalla spiaggia selezionata. Se l'utente è un Owner, deve possedere una spiaggia attiva. La spiaggia selezionata (o posseduta dall'Owner) deve essere nello stato ATTIVO.
Postcondizioni	Viene creato un nuovo record di prenotazione nel database con un prezzo totale calcolato e uno stato iniziale (PENDING per i Customer, CONFIRMED per gli Owner).
Flusso Principale	1. Il Customer seleziona una spiaggia (partendo da UC-04) e sceglie una data; 2. Il sistema verifica che il Customer non sia bannato da quella spiaggia; 3. Il sistema mostra la mappa o la lista delle disponibilità per la data scelta; 4. Il Customer seleziona i posti spiaggia desiderati (es. ombrelloni, lettini); 5. Il Customer seleziona eventuali extra (lettini, sdraio, sedie); 6. Il Customer inserisce il numero di parcheggi desiderati; 7. Il sistema calcola e mostra il prezzo totale riepilogativo; 8. Il Customer conferma la prenotazione; 9. Il sistema salva la prenotazione e le assegna lo stato "PENDING", mostrando un messaggio di successo.
Flussi alternativi	1a. Flusso Owner (Prenotazione Telefonica): i. L'Owner accede al pannello gestionale della propria spiaggia e avvia una nuova prenotazione; ii. L'Owner seleziona la data, i posti, gli extra e i parcheggi (come nei passi 3-6 del Main Flow); iii. L'Owner inserisce obbligatoriamente il Nome e il Numero di Telefono del cliente chiamante; iv. Il sistema calcola e mostra il totale; v. L'Owner conferma la prenotazione; vi. Il sistema salva la prenotazione assegnandole automaticamente lo stato "CONFIRMED". 2a-e. Utente Bannato: Il sistema rileva che il Customer è nella blacklist della spiaggia e blocca l'operazione mostrando il messaggio: "Non sei autorizzato a prenotare in questa struttura." 4/6a-e. Disponibilità Esaurita: Se durante la selezione i posti o i parcheggi scelti vengono esauriti (es. prenotati da un altro utente nello stesso istante), il sistema avvisa l'utente e lo invita a modificare la scelta.
Business Rules	• Le prenotazioni dei Customer partono sempre dallo stato PENDING e richiedono un passaggio a CONFIRMED da parte dell'Owner. • Le prenotazioni create dall'Owner sono considerate già confermate (CONFIRMED). • L'Owner non può selezionare la spiaggia: il sistema forza la prenotazione solo sullo stabilimento di sua proprietà. • Il calcolo del totale si basa sul tariffario (Season/Zone) impostato per quella data.

Figura 3: UC: Create Booking

3.2.3 Manage Reviews

ID Use Case	UC-07
Nome Use Case	Manage Reviews
Attori	Customer
Descrizione	Permette ad un Customer di valutare e commentare la propria esperienza presso una spiaggia in cui ha effettivamente soggiornato, oppure di eliminare una propria recensione precedentemente pubblicata.
Precondizioni	L'utente è loggato. La spiaggia è in stato ATTIVO. L'utente non ha un Ban per quella spiaggia. Deve esistere almeno una prenotazione per quella spiaggia con data nel passato (rispetto a quando viene applicato lo Use Case) e in stato CONFIRMED.
Postcondizioni	La recensione viene salvata o rimossa dal database, associata alla spiaggia e all'utente.
Flusso Principale	1. L'utente naviga nella sezione "Le mie prenotazioni" o sulla pagina della spiaggia; 2. L'utente clicca sul pulsante "Lascia una recensione" per la spiaggia selezionata; 3. Il sistema verifica che tutti i requisiti siano soddisfatti (prenotazione passata e CONFIRMED, assenza di ban, spiaggia attiva); 4. Il sistema mostra il modulo della recensione; 5. L'utente seleziona una valutazione (da 1 a 5 stelle) e inserisce un commento testuale; 6. L'utente clicca su "Pubblica"; 7. Il sistema salva la recensione nel database e mostra un messaggio di conferma.
Flussi alternativi	2a. Eliminazione Recensione: i. L'utente seleziona una recensione che ha già pubblicato in passato e clicca su "Elimina"; ii. Il sistema chiede conferma dell'operazione ("Sei sicuro di voler eliminare questa recensione?"); iii. L'utente conferma; iv. Il sistema rimuove il record dal database e mostra un messaggio di successo. 3a-e. Mancanza Requisiti: Se l'utente tenta di accedere all'URL della recensione senza averne diritto (es. prenotazione futura, prenotazione CANCELLED/REJECTED o mai stato in quella spiaggia), il sistema blocca l'azione mostrando: "Devi aver completato un soggiorno in questa struttura per poter lasciare una recensione." 3b-e. Spiaggia Disattivata/Ban: Se la spiaggia non è più attiva o l'utente è stato bannato da quella struttura dopo il soggiorno, il sistema inibisce la funzione di recensione. 6a-e. Recensione non completata: Se l'utente non compila entrambi i campi, il sistema evidenzia i campi da compilare obbligatoriamente.
Business Rules	• La valutazione è obbligatoria ed è un valore intero tra 1 e 5. • Il commento testuale è anch'esso obbligatorio. • Il sistema accetta recensioni solo per prenotazioni la cui data è strettamente minore della data odierna (es. prenotazione il 19 luglio, recensibile dal 20 luglio). • L'App Ban è già gestito a monte dal Login (UC-01).

Figura 4: UC: Manage Reviews

3.2.4 Create Beach

ID Use Case	UC-10
Nome Use Case	Create Beach
Attori	Owner
Descrizione	Permette a un Owner appena registrato (o senza strutture associate) di inserire nel sistema la propria spiaggia, configurandone dati anagrafici, inventario, servizi, parcheggi, stagioni e zone.
Precondizioni	L'utente deve aver effettuato il login come Owner. Nel database NON deve esistere alcuna spiaggia già associata a questo account.
Postcondizioni	Viene creato un nuovo record "Spiaggia" nel database, associato all'Owner. Lo stato sarà ATTIVO se configurata completamente e confermata, altrimenti INATTIVO (bozza).
Flusso Principale	1. L'Owner accede al sistema; il sistema rileva l'assenza di strutture e mostra il pulsante "Crea Spiaggia"; 2. L'Owner clicca sul pulsante e accede al modulo di creazione; 3. L'Owner inserisce i Dati Obbligatori: Nome, Descrizione, Numero di Telefono, Indirizzo e Informazioni Extra; 4. L'Owner (opzionalmente) compila l'Inventario: quantità di ombrelloni, tende, sdraio, lettini, sedie, camerini; 5. L'Owner (opzionalmente) seleziona i Servizi: bagni, docce, piscina, bar, ristorante, Wi-Fi, campo da pallavolo; 6. L'Owner (opzionalmente) configura il Parcheggio: numero posti auto, moto, bici, veicoli elettrici e presenza telecamere di videosorveglianza; 7. L'Owner (opzionalmente) crea le Stagioni (prezzi base e tariffe sulle varie zone) e le Zone (layout ombrelloni/tende); 8. L'Owner clicca su "Salva Spiaggia"; 9. Il sistema convalida i dati obbligatori; 10. Il sistema verifica la completezza della configurazione (presenza sia dei dati obbligatori che di quelli opzionali/operativi); 11. Avendo inserito tutti i dati, il sistema mostra un prompt: "Vuoi attivare la spiaggia e renderla visibile ora?"; 12. L'Owner accetta (o rifiuta); 13. Il sistema salva definitivamente la spiaggia nel database (in stato ATTIVO se l'Owner ha accettato, altrimenti INATTIVO).
Flussi alternativi	9a-e. Campi Obbligatori Mancanti: Se Nome, Indirizzo o altri campi base sono vuoti, il sistema evidenzia gli errori e impedisce il salvataggio. 10a. Salvataggio Parziale (Bozza): Se l'Owner ha inserito i dati obbligatori ma ha saltato configurazioni chiave (es. Stagioni e Zone), il sistema non mostra il prompt di attivazione. Salva direttamente la spiaggia in stato INATTIVO. L'Owner dovrà usare "Manage Beach" (UC-11) in futuro per completarla e attivarla.
Business Rules	• Relazione 1:1 rigida: un Owner può possedere e gestire al massimo una spiaggia nel sistema. • Per poter passare allo stato ATTIVO, la spiaggia deve possedere almeno i requisiti minimi operativi (Inventario, Servizi e Parcheggi definiti; almeno una Stagione e una Zona configurate, come definito dalle regole di business generali).

Figura 5: UC: Create Beach

3.2.5 Manage Beach

ID Use Case	UC-11
Nome Use Case	Manage Beach
Attori	Owner
Descrizione	Permette all'Owner di aggiornare i dati generali, l'inventario, i servizi, i parcheggi e di gestire la struttura delle Stagioni e delle Zone, nel rigoroso rispetto dello stato di attività della spiaggia.
Precondizioni	L'Owner ha effettuato il login, possiede una spiaggia associata e l'account/spiaggia non sono soggetti a Ban (già verificato al login).
Postcondizioni	Il database viene aggiornato con le nuove informazioni della spiaggia, dell'inventario, dei servizi e/o dei parcheggi.
Flusso Principale	1. L'Owner accede alla dashboard di Gestione Spiaggia; 2. Il sistema recupera i dati attuali e verifica lo stato della spiaggia (ATTIVO o INATTIVO); 3. Il sistema "blocca" (sola lettura) o abilita i campi di modifica in base allo stato attuale; 4. L'Owner naviga tra le sezioni e applica le modifiche desiderate (Anagrafica, Inventario, Servizi, Parcheggi); 5. L'Owner clicca su "Salva Modifiche"; 6. Il sistema esegue i controlli di integrità (es. validazione dell'inventario contro le esigenze delle stagioni future e capienza parcheggi per le prenotazioni attive); 7. Il sistema salva i dati aggiornati nel database e mostra un messaggio di successo.
Flussi alternativi	3a. Spiaggia in stato ATTIVO (Restrizioni): Il sistema rende in sola lettura: Nome, Descrizione, Numero di Telefono, Indirizzo, Inventario, Servizi e Parcheggio. Non è permessa alcuna alterazione strutturale finché la spiaggia riceve prenotazioni. 3b. Spiaggia in stato INATTIVO (Permessi Estesi): Il sistema sblocca la modifica dei dati base, Inventario, Servizi e Parcheggio. 6a-e. Conflitto Inventario: Se la spiaggia è INATTIVA e l'Owner riduce la quantità di inventario (ombrelloni, tende), il sistema verifica le stagioni attuali e future. Se la nuova capacità è inferiore a quella utilizzata nelle Stagioni, il sistema blocca il salvataggio mostrando: "Inventario insufficiente: capacità ridotta sotto la soglia richiesta dalle stagioni programmate." 6b-e. Conflitto Parcheggi: Se l'Owner riduce il numero di parcheggi, il sistema somma i posti auto richiesti da tutte le prenotazioni PENDING e CONFIRMED per ogni singola data (odierna e futura). Se il nuovo limite inserito è inferiore al totale già prenotato in una qualsiasi di queste date, il sistema blocca l'operazione mostrando: "Capacità parcheggio insufficiente per coprire le prenotazioni già attive o in attesa."
Business Rules	• Dati anagrafici, Inventario, Servizi e Parcheggio sono modificabili esclusivamente in stato INATTIVO. • I controlli di capienza (Inventario e Parcheggi) devono garantire la retrocompatibilità sia con le Stagioni in corso e quelle in futuro che con le prenotazioni già accettate o in approvazione (CONFIRMED/PENDING).

Figura 6: UC: Manage Beach

4 Progettazione

4.1 Pattern Architetture e Progettuali

Per lo sviluppo di SunSpot sono stati adottati diversi pattern volti a garantire un'architettura robusta, testabile e facilmente manutenibile, separando nettamente le responsabilità tra logica di business e dettagli implementativi.

4.1.1 Architettura Esagonale (Ports & Adapters)

Il progetto SunSpot adotta i principi dell'Architettura Esagonale per garantire una separazione chirurgica delle responsabilità e un totale isolamento della logica di business dalle tecnologie esterne. L'architettura prevede un nucleo centrale (**Core**), composto dai package **domain** e **application**, e un perimetro esterno composto dagli **Adapters** nel package **infrastructure**.

Lo strato di **Application** funge da ponte tra il mondo esterno e il dominio, ed è suddiviso in due componenti fondamentali:

- **Ports (Interfacce)**: rappresentano i contratti di comunicazione dell'esagono. Si distinguono in:
 - **Port-In (Driving Ports)**: definiscono i casi d'uso e le azioni che gli attori esterni possono richiedere al sistema (es. elenco 1).
 - **Port-Out (Driven Ports)**: definiscono i requisiti infrastrutturali necessari al core per funzionare (es. elenco 2).
- **Services (Orchestratori)**: rappresentano le implementazioni concrete delle Port-In. Il loro ruolo è quello di **orchestratori dei casi d'uso**: un Service riceve un comando, recupera i dati necessari tramite le Port-Out, delega la logica decisionale e di validazione alle entità del **domain** e infine persiste i cambiamenti tramite le Port-Out. Il Service non contiene logica di business, ma si occupa esclusivamente di coordinare il flusso operativo e gestire l'integrità delle transazioni.

Grazie a questa struttura, il sistema gode di una straordinaria resilienza: l'utilizzo delle interfacce assicura che il **disaccoppiamento** sia totale. Nel nostro caso, l'adattatore **InfrastructureJdbc** contiene il codice specifico per interagire con il database MySQL; se in futuro fosse necessario migrare verso un diverso sistema di persistenza o un servizio cloud, sarebbe sufficiente sviluppare un nuovo Adapter senza dover modificare una singola riga di codice nel Core applicativo.

Listing 1: Esempio Port-In

Listing 2: Esempio Port-Out

Listing 3: Esempio Service

4.1.2 DDD Light (Domain-Driven Design)

A integrazione dell'architettura esagonale, è stato adottato l'approccio **DDD Light**. Mentre l'esagonale definisce i confini, il DDD stabilisce che il nucleo del progetto deve riflettere il linguaggio e le regole della realtà modellata.

Il cuore di questo approccio è il **Rich Domain Model**: a differenza di un modello anemico (fatto di soli getter e setter), nel nostro dominio le entità hanno il compito di assicurarsi l'integrità dei dati sia in fase di modifica che di istanziazione. Ad esempio, la classe **Beach** presenta metodi interni dedicati a controllare la presenza di tutti i dati obbligatori e il rispetto dei vincoli di business prima che l'oggetto possa essere effettivamente creato o modificato.

In questa architettura abbiamo distinto:

- **Entities**: oggetti con un'identità univoca che persiste nel tempo (es. elenco 4).
- **Value Objects**: oggetti definiti dai propri attributi e immutabili, implementati tramite i **Java Records** (es. elenco 5).

4.1.3 Builder Pattern

Il pattern Builder è stato utilizzato per gestire l'istanziamento di classi caratterizzate da un elevato numero di parametri e attributi opzionali, evitando costruttori eccessivamente complessi e verbosi. Tramite una classe interna statica la costruzione dell'oggetto avviene in modo guidato e leggibile (es. elenco 5).

Il pattern è stato applicato a classi come **Address**, **Spot**, **Zone** e ai record associati a **Booking** e **Beach**. Nel caso della spiaggia, vista la densità di informazioni richieste, abbiamo suddiviso gli attributi su diversi record tematici, ciascuno dei quali viene istanziato tramite il proprio builder. Questo ci permette di gestire con estrema facilità la configurazione di parametri non obbligatori, mantenendo il codice pulito e uniforme in tutto il progetto.

4.2 Struttura del codice

La struttura della directory del progetto è stata organizzata in base alla separazione dei package di Java e seguendo i principi dell'architettura software a livelli, descritta precedentemente.

```
|-- .idea
|-- .mvn
|-- db-init
|-- src
|   |-- main
|   |   |-- java
|   |   |   |-- com.example.s_balneare
|   |   |       |-- application
|   |   |       |   |-- port
|   |   |       |   |   |--in
|   |   |       |   |   |--out
|   |   |       |   |-- service
|   |   |       |-- domain
|   |   |       |-- infrastructure.persistence.jdbc
|   |   |       |-- Main.java
|   |   |       |-- module-info.java
|   |   |-- resources
|-- test
|   |-- java
|       |-- com.example.s_balneare
|           |-- application
|           |   |--service
|           |-- domain
|           |-- AllTestsSuite.java
|           |-- ApplicationTestSuite.java
|           |-- DomainTestSuite.java
|-- target
|-- .gitignore
|-- docker-compose.yml
```

La directory dei test rispecchia la struttura principale per mantenere gli unit test organizzati (con package paralleli come **application** e **domain** e le relative Test Suite). L'adozione di queste pratiche garantisce un progetto chiaro, strutturato e facilmente manutenibile, evitando al contempo problemi legati alla visibilità dei package.

Ogni package interagisce con gli altri per svolgere compiti specifici, mantenendo una netta separazione tra il dominio applicativo (**domain**) , i casi d'uso (**application**) e la gestione dei dati e interazioni con il database (**infrastructure**). Questo approccio strutturato assicura che ogni componente del sistema abbia una responsabilità ben definita, rendendo il codice più facile da navigare, comprendere ed estendere. Inoltre, facilita la collaborazione tra i membri del team, poiché ogni package può essere sviluppato e testato in modo indipendente.

4.3 Domain

4.4 Application

4.5 Infrastructure

5 Database

Per la gestione della persistenza dei dati è stato utilizzato il DBMS **MySQL**, configurato all'interno di un ambiente di containerizzazione **Docker**. L'interazione tra l'applicazione Java e il database avviene tramite l'interfaccia **JDBC**, utilizzando il driver ufficiale **MySQL Connector/J**.

L'integrità del dato è assicurata a livello nativo attraverso una rigorosa definizione di vincoli (Foreign Key, Unique e Check) implementati nello script di creazione. Tale approccio permette di delegare al database la validazione delle informazioni critiche, mentre le verifiche legate alla **logica di business** sono gestite programmaticamente all'interno del codice Java dell'applicazione.

5.1 Modello ER

Il modello Entity-Relationship (fig. 7) omette la relazione diretta tra *Beaches* e *Bookings* per evitare ridondanze concettuali, essendo il legame già mediato dalle entità *Zones* e *Spots*. In fase di implementazione del database fisico, tuttavia, tale relazione è stata introdotta per ottimizzare le performance delle query di ricerca, riducendo la complessità delle operazioni di join.

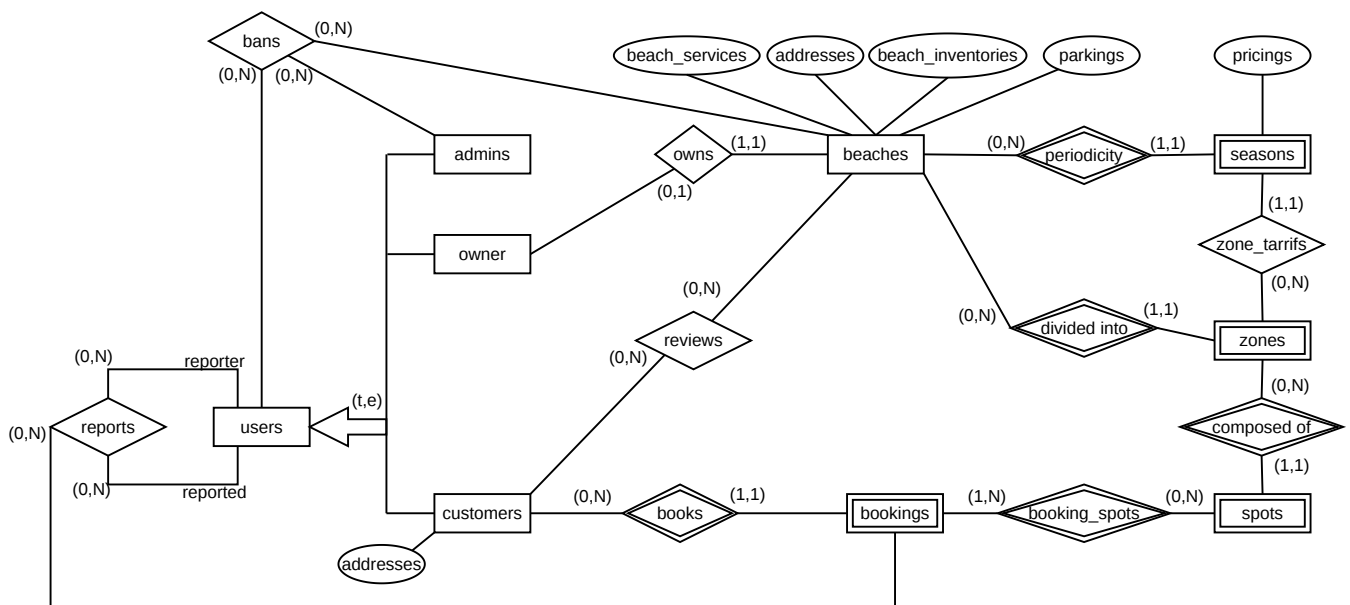


Figura 7: Modello Entity Relationship (Schema Concettuale)

5.2 Dizionario dei dati

Il dizionario (fig. 8 e 9) presenta per *Spots* e *Bookings* attributi identificativi esterni derivanti dalle relazioni con altre entità. Per semplificare le interrogazioni e la realizzazione del database si è optato per l'utilizzo di una chiave primaria artificiale (*id*), preservando la valenza identificativa di questi attributi tramite vincoli **UNIQUE** e **NOT NULL**, come dettagliato nella tabella dei vincoli (fig. 10).

Entità	Descrizione	Attributi	ID
Users	Generalizzazione degli utenti	name, surname, username, email, hashPassword	id
Admins	Amministratori di sistema	OTP	id (<i>Users</i>)
Owners	Gestori degli stabilimenti	active, OTP	id (<i>Users</i>)
Customers	Utenti (Clienti)	phoneNumber, active	id (<i>Users</i>)
Beaches	Stabilimenti balneari	name, description, phoneNumber, extraInfo, active, closed	id
Parkings	Attributo composto di Beaches. Info parcheggio	nAutoPark, nMotoPark, nElectricPark, CCTV	-
Beach Services	Attributo composto di Beaches. Servizi offerti	bathrooms, showers, pool, bar, restaurant, wifi	-
Beach Inventories	Attributo composto di Beaches. Inventario	countExtraSdraio, countExtraLettini, countExtraSedie, countCamerini	-
Addresses	Attributo composto. Indirizzi di utenti e spiagge	street, streetNumber, city, zipCode, country	-
Pricings	Listino prezzi base	priceLettino, priceSdraio, priceSedia, priceParking	-
Seasons	Periodi temporali (Stagionalità)	startDate, endDate, name	name, beachId (<i>Beaches</i>)
Zones	Aree geografiche della spiaggia	name	name, beachId (<i>Beaches</i>)
Spots	Postazioni prenotabili	type, row, column, zoneName (<i>Zones</i>), beachId (<i>Beaches</i>)	id
Bookings	Dati della prenotazione	date, extraSdraio, extraLettini, extraSedie, camerini, autoPark, motoPark, electricPark, totalPrice, status, callerName, callerPhone, customerId (<i>Customers</i>), beachId (<i>Beaches</i>)	id

Figura 8: Dizionario delle Entità

Relazioni	Descrizione	Entità coinvolte	Attributi
Bans	Gestione allontanamenti	Admins (0,N), Users (0,N), Beaches (0,N)	reason, banType, createdAt
Owns	Proprietà dello stabilimento	Owners (0,1), Beaches (1,1)	-
Reviews	Valutazioni degli utenti	Customers (0,N), Beaches (0,N)	rating, comment, createdAt
Books	Assegnazione prenotazione	Customers (0,N), Bookings (1,1)	-
Periodicity	Associazione stagionale	Beaches (0,N), Seasons (1,1)	-
Zone Tariffs	Prezzi per zona specifica	Seasons (1,1), Zones (0,N)	priceOmbrellone, priceTenda
Divided into	Partizione della spiaggia	Beaches (0,N), Zones (1,1)	-
Composed of	Layout dei posti	Zones (0,N), Spots (1,1)	-
Booking Spots	Occupazione fisica spot	Bookings (1,N), Spots (0,N)	date
Reports	Segnalazioni problemi	Users (0,N), Bookings (1,1)	status, description, createdAt

Figura 9: Dizionario delle Relazioni

5.3 Tabelle delle Regole

Di seguito vengono riportati i vincoli di integrità e le regole di derivazione che governano la logica della persistenza dei dati. In merito alla regola RD1 (fig. 11), si è scelta la persistenza dell'attributo *totalPrice* nel database al fine di ridurre il carico computazionale di consultazione.

Codice	Regola di Vincolo (RV)
RV1	L'email e lo username di un utente (<i>Users</i>) devono essere univoci a livello di sistema.
RV2	Il numero di telefono (<i>phoneNumber</i>) di un cliente deve essere univoco.
RV3	Ogni spiaggia (<i>Beaches</i>) deve riferirsi obbligatoriamente a un proprietario (<i>Owners</i>) e a un indirizzo (<i>Addresses</i>) esistenti in modo univoco.
RV4	Il numero di posti auto, moto ed elettrici (<i>Parkings</i>) deve essere maggiore o uguale a zero.
RV5	Le quantità dell'inventario (<i>beach_inventories</i>), come ombrelloni e lettini, devono essere maggiori o uguali a zero.
RV6	Tutti i prezzi definiti nei listini (<i>Pricings</i>) e nelle tariffe per zona (<i>zone_tariffs</i>) devono essere maggiori o uguali a zero.
RV7	La data di inizio di una stagione (<i>startDate</i>) deve essere strettamente precedente alla data di fine (<i>endDate</i>).
RV8	Non possono esistere due postazioni (<i>Spots</i>) con la stessa riga e colonna nella medesima zona e spiaggia.
RV9	Le coordinate di riga (<i>row</i>) e colonna (<i>column</i>) di uno <i>Spot</i> devono essere maggiori o uguali a zero.
RV10	Un cliente non può effettuare più di una prenotazione (<i>Bookings</i>) per la stessa spiaggia nella medesima data.
RV11	Uno <i>Spot</i> non deve essere associato a più di una prenotazione (<i>booking_spots</i>) nella stessa data.
RV12	Le quantità di servizi extra richiesti in una prenotazione (lettini, sdraio, ecc.) devono essere maggiori o uguali a zero.
RV13	Un utente non può ricevere più di un ban attivo (<i>Bans</i>) per la stessa spiaggia o per l'intera applicazione.
RV14	Il rating di una recensione (<i>Reviews</i>) deve essere un valore intero compreso tra 1 e 5.
RV15	Un cliente può inserire al massimo una recensione per ogni spiaggia.
RV16	Ogni utente può emettere un solo report (<i>Reports</i>) per una specifica prenotazione.
RV17	Utilizzo identificatore surrogato (id) per Spots, deve esistere vincolo di unicità tra: <i>row</i> , <i>column</i> , <i>zoneName</i> e <i>beachId</i> .
RV18	Utilizzo identificatore surrogato (id) per Bookings, deve esistere vincolo di unicità tra: <i>customerId</i> , <i>beachId</i> e <i>date</i> .
RV19	La spiaggia associata a una prenotazione (<i>beachId</i> in <i>Bookings</i>) deve corrispondere alla spiaggia a cui appartengono i singoli posti prenotati (<i>beachId</i> in <i>Spots</i>).

Figura 10: Tabella dei vincoli di integrità (RV)

Codice	Regola di Derivazione (RD)
RD1	Il prezzo totale (<i>totalPrice</i>) di una prenotazione si ottiene sommando la tariffa dello spot (basata su zona e stagione) al costo dei servizi extra e dei parcheggi prenotati (quantità moltiplicata per il rispettivo prezzo unitario).

Figura 11: Tabella delle regole di derivazione (RD)

6 Mockup

7 Testing